

**Министерство науки и высшего образования Российской
Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ ГОСУДАРСТВЕННОЕ
АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
ИТМО**

ОТЧЕТ

по Лабораторной работе № 4

**«Запросы на выборку и модификацию данных.
Представления. Работа с индексами.»**

**по дисциплине «Проектирование и реализация баз
данных»**

Обучающийся: Петрова Мария Валерьевна

Факультет: прикладной информатики

Группа: К3239

Направление подготовки: 09.03.03 Прикладная
информатика

Образовательная программа: Мобильные и сетевые
технологии 2023

Преподаватели: Говорова Марина Михайловна
Белов Александр

Санкт-Петербург

2025 г.

1 Цель работы

Овладеть практическими навыками создания представлений и запросов на выборку данных к базе данных PostgreSQL, использования подзапросов при модификации данных и индексов.

2 Практическое задание

В рамках лабораторной работы требовалось:

1. Создать запросы и представления на выборку данных к базе данных PostgreSQL (согласно индивидуальному заданию лабораторной работы №2, часть 2 и 3);
2. Составить 3 запроса на модификацию данных (INSERT, UPDATE, DELETE) с использованием подзапросов;
3. Изучить графическое представление запросов и просмотреть историю запросов;
4. Создать простой и составной индексы для двух произвольных запросов и сравнить время выполнения запросов без индексов и с индексами. Для получения плана запроса использовать команду EXPLAIN ANALYZE.

3 Выполнение работы

3.1 Наименование базы данных и вариант

Вариант: 1 — БД «ОТЕЛЬ».

Наименование базы данных: **hotel_db**.

Схема базы данных: **hotel**.



3.2 Запросы на выборку данных

Запрос 1. Составить список всех 2-местных номеров отелей, с ценой менее 200 т.р., упорядочив данные в порядке уменьшения стоимости

```
1  select
2      h.name as hotel_name,
3      r.room_number,
4      rt.name as room_type,
5      p.price_value
6  from hotel.room r
7  join hotel.hotel h on h.id = r.hotel_id
8  join hotel.room_type rt on rt.id = r.room_type_id
9  join hotel.price p on p.room_type_id = rt.id
10 where rt.name = 'двухместный'
11      and p.price_value < 200000
12      and p.start_date <= current_date
13      and p.end_date >= current_date
14 order by p.price_value desc;
```

Data Output Messages Notifications

	hotel_name character varying (60)	room_number integer	room_type character varying (30)	price_value integer
1	Отель Волга	112	двухместный	6200
2	Отель Волга	106	двухместный	6200
3	Отель Волга	105	двухместный	6200
4	Отель Волга	103	двухместный	6200
5	Отель Невский	102	двухместный	6200

Рис. 2: Результат выполнения запроса 1

В результате был получен список всех двухместных номеров отелей с актуальной стоимостью менее 200000 рублей, отсортированный по убыванию цены. Были найдены номера в отелях «Отель Волга» и «Отель Невский».

Запрос 2. Выбрать все записи регистрации постояльцев, которые выехали из отелей в течение двух последних недель

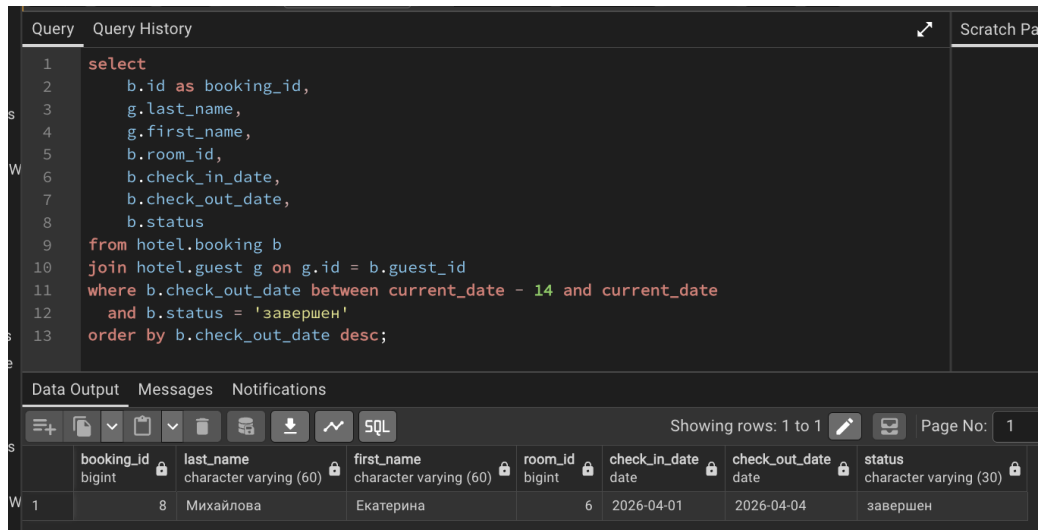


Рис. 3: Результат выполнения запроса 2

Запрос вывел записи о постояльцах, завершивших проживание за последние две недели. Была найдена запись о выезде Михайловой Екатерины.

Запрос 3. Определить общий суточный доход каждого отеля за последний месяц

Query	Query History
1	<code>select</code>
2	<code> h.name as hotel_name,</code>
3	<code> sum(b.total_cost) as total_income</code>
4	<code>from hotel.booking b</code>
5	<code>join hotel.room r on r.id = b.room_id</code>
6	<code>join hotel.hotel h on h.id = r.hotel_id</code>
7	<code>where b.booking_date between current_date - 30 and current_date</code>
8	<code> and b.status in ('завершен', 'заселен')</code>
9	<code>group by h.name</code>
10	<code>order by total_income desc;</code>

Data Output	Messages	Notifications															
≡+ 📄 ▼ 📋 ▼ 🗑️ 🗄️ ⬇️ 📈 SQL Show																	
	<table><thead><tr><th></th><th>hotel_name character varying (60)</th><th>total_income bigint</th></tr></thead><tbody><tr><td>1</td><td>Отель Сочи Резорт</td><td>55500</td></tr><tr><td>2</td><td>Отель Московские Ог...</td><td>43500</td></tr><tr><td>3</td><td>Отель Волга</td><td>31000</td></tr><tr><td>4</td><td>Отель Невский</td><td>12600</td></tr></tbody></table>		hotel_name character varying (60)	total_income bigint	1	Отель Сочи Резорт	55500	2	Отель Московские Ог...	43500	3	Отель Волга	31000	4	Отель Невский	12600	
	hotel_name character varying (60)	total_income bigint															
1	Отель Сочи Резорт	55500															
2	Отель Московские Ог...	43500															
3	Отель Волга	31000															
4	Отель Невский	12600															

Рис. 4: Результат выполнения запроса 3

В результате был рассчитан суммарный доход каждого отеля за последний месяц. Наибольший доход показал «Отель Сочи Резорт».

Запрос 4. Составить список свободных номеров заданного отеля на текущий день

```
1  select
2      h.name as hotel_name,
3      r.room_number,
4      rt.name as room_type,
5      r.occupancy_status
6  from hotel.room r
7  join hotel.hotel h on h.id = r.hotel_id
8  join hotel.room_type rt on rt.id = r.room_type_id
9  where h.name = 'Отель Волга'
10     and r.occupancy_status = 'свободен'
11  order by r.room_number;
```

Data Output Messages Notifications

SQL Showing

	hotel_name character varying (60) 🔒	room_number integer 🔒	room_type character varying (30) 🔒	occupancy_status character varying (30) 🔒
1	Отель Волга	103	двухместный	свободен
2	Отель Волга	104	семейный	свободен
3	Отель Волга	105	двухместный	свободен
4	Отель Волга	106	двухместный	свободен
5	Отель Волга	108	одноместный	свободен
6	Отель Волга	110	трехместный	свободен
7	Отель Волга	112	двухместный	свободен

Рис. 5: Результат выполнения запроса 4

Запрос вывел список свободных номеров для отеля «Отель Волга». В результат были включены номер комнаты, тип номера и текущий статус занятости.

Запрос 5. Найти общие потери от незанятых номеров за текущий день по всей сети

The screenshot shows a database query editor with two tabs: 'Query' and 'Query History'. The 'Query' tab is active, displaying an SQL query. Below the query, there are tabs for 'Data Output', 'Messages', and 'Notifications'. The 'Data Output' tab is active, showing a table with the results of the query. The table has two columns: 'total_loss' (bigint) and a lock icon. The first row shows a value of 97300.

```
1 select
2     sum(p.price_value) as total_loss
3 from hotel.room r
4 join hotel.room_type rt on rt.id = r.room_type_id
5 join hotel.price p on p.room_type_id = rt.id
6 where r.occupancy_status = 'свободен'
7     and p.start_date <= current_date
8     and p.end_date >= current_date;
```

	total_loss bigint
1	97300

Рис. 6: Результат выполнения запроса 5

По результатам была рассчитана суммарная потеря дохода от свободных номеров по всей сети за текущий день. Она составила 97300 рублей.

Запрос 6. Определить, в каком отеле имеется наибольшее количество незанятых номеров на текущие сутки

The screenshot shows a SQL IDE interface. The top panel, titled 'Query', contains a SQL query. The bottom panel, titled 'Data Output', shows the result of the query in a table format. The table has two columns: 'hotel_name' and 'free_room_count'. The result shows one row with the hotel name 'Отель Волга' and a free room count of 7.

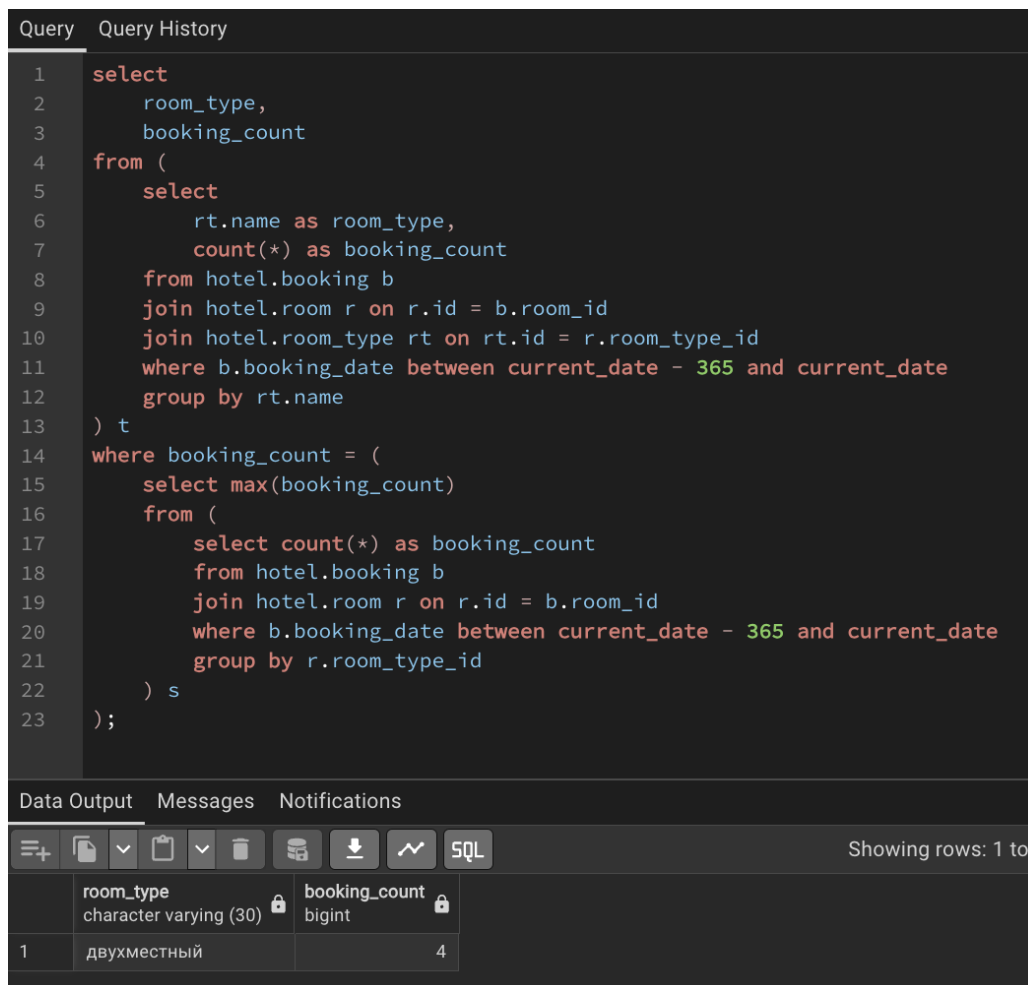
```
1  select
2      hotel_name,
3      free_room_count
4  from (
5      select
6          h.name as hotel_name,
7          count(*) as free_room_count
8      from hotel.room r
9      join hotel.hotel h on h.id = r.hotel_id
10     where r.occupancy_status = 'свободен'
11     group by h.name
12 ) t
13 where free_room_count = (
14     select max(free_room_count)
15     from (
16         select count(*) as free_room_count
17         from hotel.room r
18         where r.occupancy_status = 'свободен'
19         group by r.hotel_id
20     ) s
21 );
```

	hotel_name character varying (60)	free_room_count bigint
1	Отель Волга	7

Рис. 7: Результат выполнения запроса 6

«Отель Волга» имеет максимальное количество свободных номеров - 7 номеров.

Запрос 7. Определить самый популярный тип номеров за последний год



```
1 select
2     room_type,
3     booking_count
4 from (
5     select
6         rt.name as room_type,
7         count(*) as booking_count
8     from hotel.booking b
9     join hotel.room r on r.id = b.room_id
10    join hotel.room_type rt on rt.id = r.room_type_id
11    where b.booking_date between current_date - 365 and current_date
12    group by rt.name
13 ) t
14 where booking_count = (
15     select max(booking_count)
16     from (
17         select count(*) as booking_count
18         from hotel.booking b
19         join hotel.room r on r.id = b.room_id
20         where b.booking_date between current_date - 365 and current_date
21         group by r.room_type_id
22     ) s
23 );
```

Data Output Messages Notifications

Showing rows: 1 to 1

	room_type character varying (30)	booking_count bigint
1	двухместный	4

Рис. 8: Результат выполнения запроса 7

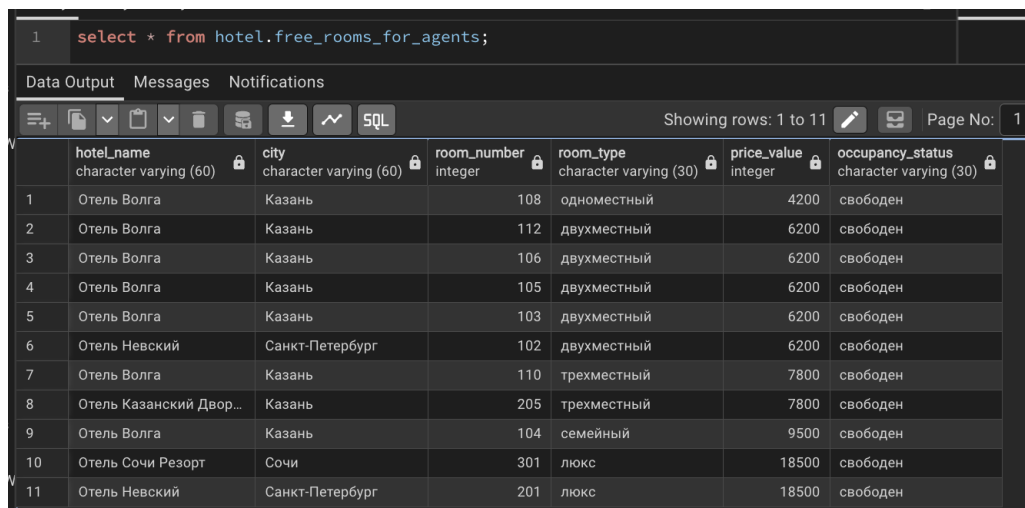
Самым популярным типом номера за последний год является двухместный. Количество бронирований данного типа номера - 4.

3.3 Создание представлений

Представление 1. Для турагентов (поиск свободных номеров в отелях)

```
1  create view hotel.free_rooms_for_agents as
2  select
3      h.name as hotel_name,
4      h.city,
5      r.room_number,
6      rt.name as room_type,
7      p.price_value,
8      r.occupancy_status
9  from hotel.room r
10 join hotel.hotel h on h.id = r.hotel_id
11 join hotel.room_type rt on rt.id = r.room_type_id
12 join hotel.price p on p.room_type_id = rt.id
13 where r.occupancy_status = 'свободен'
14        and p.start_date <= current_date
15        and p.end_date >= current_date;
```

Рис. 9: Создание представления free_rooms_for_agents



	hotel_name character varying (60)	city character varying (60)	room_number integer	room_type character varying (30)	price_value integer	occupancy_status character varying (30)
1	Отель Волга	Казань	108	одноместный	4200	свободен
2	Отель Волга	Казань	112	двухместный	6200	свободен
3	Отель Волга	Казань	106	двухместный	6200	свободен
4	Отель Волга	Казань	105	двухместный	6200	свободен
5	Отель Волга	Казань	103	двухместный	6200	свободен
6	Отель Невский	Санкт-Петербург	102	двухместный	6200	свободен
7	Отель Волга	Казань	110	трехместный	7800	свободен
8	Отель Казанский Двор...	Казань	205	трехместный	7800	свободен
9	Отель Волга	Казань	104	семейный	9500	свободен
10	Отель Сочи Резорт	Сочи	301	люкс	18500	свободен
11	Отель Невский	Санкт-Петербург	201	люкс	18500	свободен

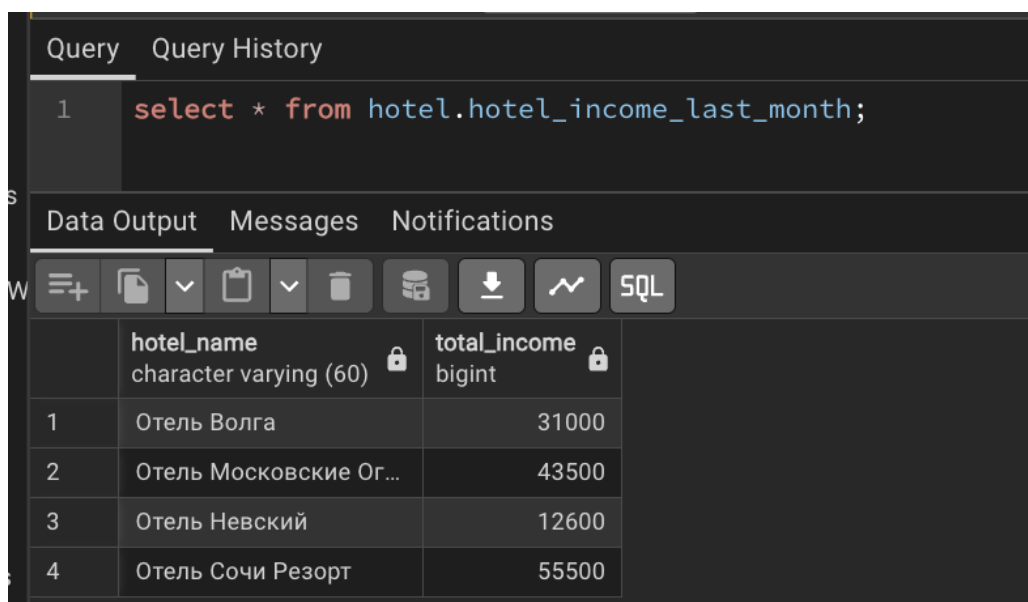
Рис. 10: Результат проверки представления free_rooms_for_agents

Представление free_rooms_for_agents для турагентов позволяет получить информацию о свободных номерах в отелях сети. В результате отображаются название отеля, город, номер комнаты, тип номера, актуальная стоимость и статус занятости.

Представление 2. Для владельца компании (информация о доходах каждого отеля в сети за прошедший месяц)

```
Query  Query History
1  create view hotel.hotel_income_last_month as
2  select
3      h.name as hotel_name,
4      sum(b.total_cost) as total_income
5  from hotel.booking b
6  join hotel.room r on r.id = b.room_id
7  join hotel.hotel h on h.id = r.hotel_id
8  where b.booking_date between current_date - 30 and current_date
9        and b.status in ('завершен', 'заселен')
10 group by h.name;
```

Рис. 11: Создание представления hotel_income_last_month



The screenshot shows a database interface with a 'Query' tab and a 'Query History' tab. The 'Query' tab contains the following SQL statement:

```
1  select * from hotel.hotel_income_last_month;
```

Below the query, there are tabs for 'Data Output', 'Messages', and 'Notifications'. The 'Data Output' tab is active, showing a table with two columns: 'hotel_name' (character varying (60)) and 'total_income' (bigint). The table contains four rows of data:

	hotel_name	total_income
1	Отель Волга	31000
2	Отель Московские Ог...	43500
3	Отель Невский	12600
4	Отель Сочи Резорт	55500

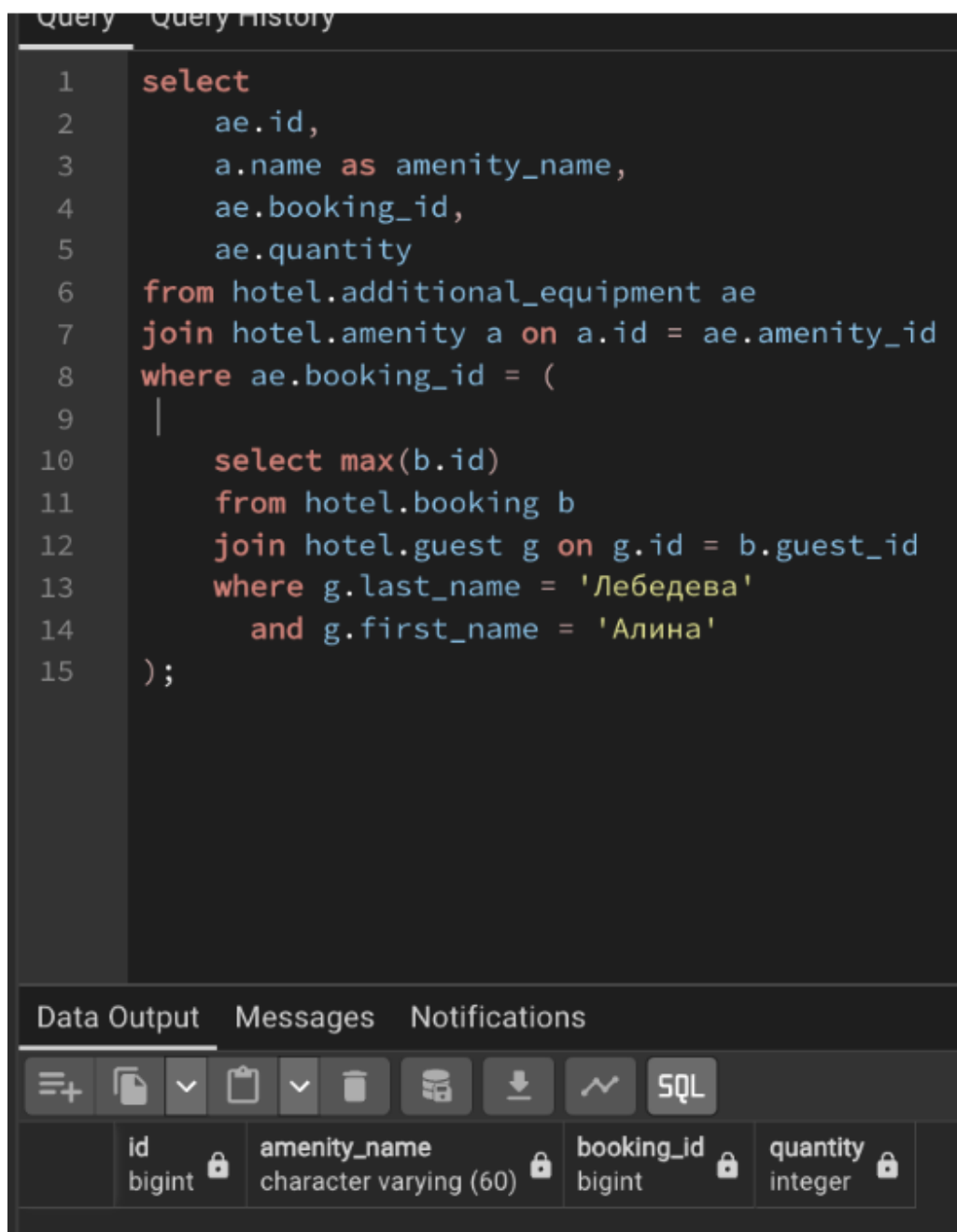
Рис. 12: Результат проверки представления hotel_income_last_month

Представление `hotel_income_last_month` для владельца компании отображает суммарный доход каждого отеля в сети за последний месяц. В результате выполнения представлены названия отелей и рассчитанный доход по каждому из них.

3.4 Запросы на модификацию данных с использованием подзапросов

Запрос INSERT

Формулировка: добавить в таблицу `additional_equipment` услугу «Завтрак» для последнего бронирования гостя Лебедева Алина с использованием подзапроса.



The screenshot shows a SQL IDE interface. The top pane displays a SQL query with line numbers 1 through 15. The query selects columns from the `additional_equipment` table, joining it with the `amenity` table and a subquery that finds the latest booking for a guest named 'Алина Лебедева'. The bottom pane shows the 'Data Output' tab with a table of results.

```
1 select
2     ae.id,
3     a.name as amenity_name,
4     ae.booking_id,
5     ae.quantity
6 from hotel.additional_equipment ae
7 join hotel.amenity a on a.id = ae.amenity_id
8 where ae.booking_id = (
9     |
10    select max(b.id)
11    from hotel.booking b
12    join hotel.guest g on g.id = b.guest_id
13    where g.last_name = 'Лебедева'
14          and g.first_name = 'Алина'
15 );
```

Data Output			
	id	amenity_name	booking_id
	bigint	character varying (60)	bigint
			integer

Рис. 13: Состояние данных до выполнения INSERT

```
insert into hotel.additional_equipment (  
    id,  
    amenity_id,  
    booking_id,  
    quantity  
)  
values (  
    6,  
    (select id  
     from hotel.amenity  
     where name = 'Завтрак'),  
  
    (select max(b.id)  
     from hotel.booking b  
     join hotel.guest g on g.id = b.guest_id  
     where g.last_name = 'Лебедева'  
           and g.first_name = 'Алина'),  
    1  
);
```

Рис. 14: Выполнение запроса INSERT

```

1  select
2      ae.id,
3      a.name as amenity_name,
4      ae.booking_id,
5      ae.quantity
6  from hotel.additional_equipment ae
7  join hotel.amenity a on a.id = ae.amenity_id
8  where ae.booking_id = (
9      select max(b.id)
10     from hotel.booking b
11    join hotel.guest g on g.id = b.guest_id
12   where g.last_name = 'Лебедева'
13         and g.first_name = 'Алина'
14 );

```

Data Output Messages Notifications

	id bigint	amenity_name character varying (60)	booking_id bigint	quantity integer
1	6	Завтрак	15	1

Рис. 15: Состояние данных после выполнения INSERT

До выполнения запроса у последнего бронирования гостя Лебедева Алина отсутствовали записи в таблице `additional_equipment`. После выполнения INSERT с использованием подзапросов была успешно добавлена услуга «Завтрак».

Запрос UPDATE

Формулировка: изменить статус уборки на «убран» для записей уборки, относящихся к свободным номерам, с использованием подзапроса.

Query

Query History

1

select

2

c.id,

3

c.room_id,

4

c.cleaning_date,

5

c.status,

6

c.comment

7

from hotel.cleaning c

8

where c.room_id in (

9

select r.id

10

from hotel.room r

11

where r.occupancy_status = 'свободен'

12

)

13

order by c.id;

Data Output

Messages

Notifications

≡

📄

▼

📄

▼

🗑️

🗑️

📄

⬇️

📈

SQL

Showing rows:

	id [PK] bigint	room_id bigint	cleaning_date date	status character varying (30)	comment character varying (255)
1	2	2	2025-03-01	убран	Уборка после проживания
2	3	3	2025-03-01	не убран	Ожидает уборки
3	4	4	2025-03-02	убран	Плановая уборка
4	5	5	2025-03-02	не убран	Гость в номере, уборка отложе...
5	6	6	2025-03-03	убран	Подготовка к заезду

Рис. 16: Состояние данных до выполнения UPDATE

```

update hotel.cleaning
set status = 'убран'
where room_id in (
    select r.id
    from hotel.room r
    where r.occupancy_status = 'свободен'
)
and status = 'не убран';

```

Рис. 17: Выполнение запроса UPDATE


```
1  select
2      c.id,
3      c.room_id,
4      c.cleaning_date,
5      c.status,
6      c.comment
7  from hotel.cleaning c
8  where c.room_id in (
9      select r.id
10     from hotel.room r
11     where r.occupancy_status = 'свободен'
12 )
13 order by c.id;
```

Data Output

Messages

Notifications

≡+

📄

▼

📋

▼

🗑

🗄

⬇

📈

SQL

Showing rows:

	id [PK] bigint	room_id bigint	cleaning_date date	status character varying (30)	comment character varying (255)
1	2	2	2025-03-01	убран	Уборка после проживания
2	3	3	2025-03-01	убран	Ожидает уборки
3	4	4	2025-03-02	убран	Плановая уборка
4	5	5	2025-03-02	убран	Гость в номере, уборка отложе...
5	6	6	2025-03-03	убран	Подготовка к заезду

Рис. 18: Состояние данных после выполнения UPDATE

После выполнения запроса статус уборки для записей, относящихся к свободным номерам, был изменён на «убран».

Запрос DELETE

Формулировка: удалить из таблицы `promotion` акцию для типа номера «двухместный», срок действия которой уже закончился, с использованием подзапроса.

```

1  select
2      p.id,
3      rt.name as room_type,
4      p.conditions,
5      p.discount_percent,
6      p.start_date,
7      p.end_date
8  from hotel.promotion p
9  join hotel.room_type rt on rt.id = p.room_type_id
10 where p.id = (
11     select p2.id
12     from hotel.promotion p2
13     join hotel.room_type rt2 on rt2.id = p2.room_type_id
14     where rt2.name = 'двухместный'
15           and p2.end_date < current_date
16     order by p2.end_date
17     limit 1
18 );

```

Data Output Messages Notifications

Showing rows: 1 to 1

	id bigint	room_type character varying (30)	conditions character varying (255)	discount_percent integer	start_date date	end_date date
1	2	двухместный	Весенняя акция	15	2025-04-01	2025-05-31

Рис. 19: Состояние данных до выполнения DELETE

```

delete from hotel.promotion
where id = (
    select p2.id
    from hotel.promotion p2
    join hotel.room_type rt2 on rt2.id = p2.room_type_id
    where rt2.name = 'двухместный'
           and p2.end_date < current_date
    order by p2.end_date
    limit 1
);

```

Рис. 20: Выполнение запроса DELETE

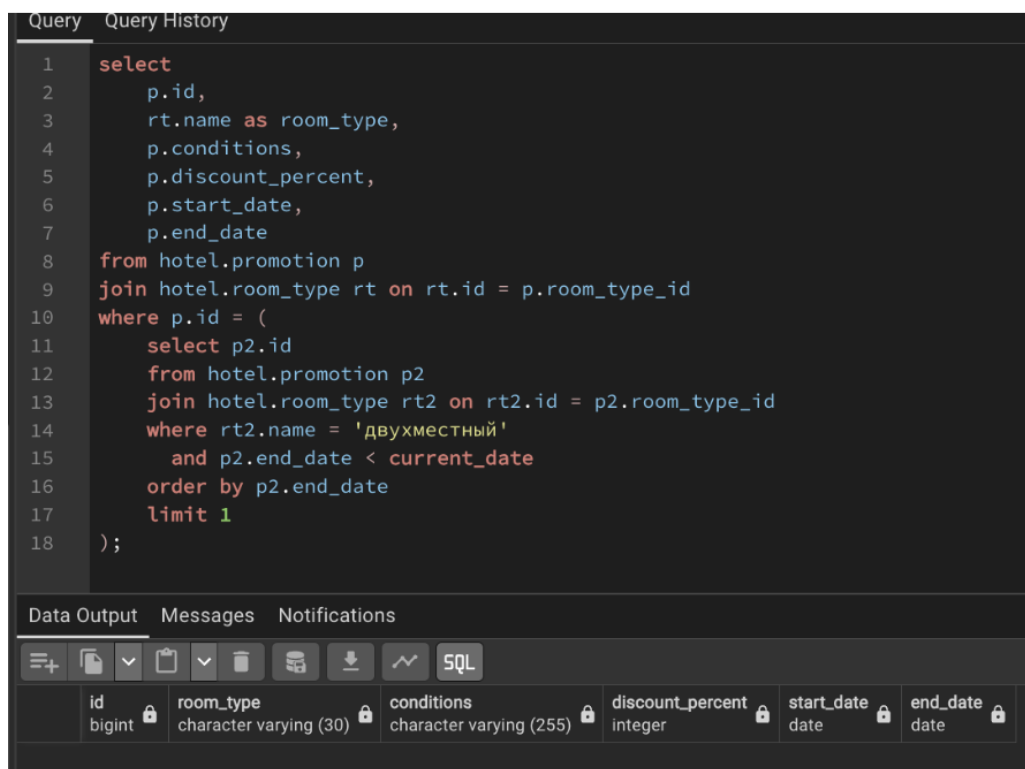


Рис. 21: Состояние данных после выполнения DELETE

После выполнения запроса устаревшая акция для двухместного номера больше не находилась по заданному условию, что подтверждает корректное удаление записи.

3.5 Создание индексов и анализ плана выполнения запросов

Для анализа влияния индексов на выполнение запросов были выбраны два запроса. Для каждого был получен план выполнения и время работы до создания индекса, после чего был создан индекс и выполнен повторный анализ.

Простой индекс для запроса поиска свободных номеров

Для анализа был выбран запрос найти все бронирования с датой выезда за последние 14 дней..

1	explain analyze
2	select
3	id,
4	room_id,
5	guest_id,
6	check_in_date,
7	check_out_date,
8	status,
9	total_cost
10	from hotel.booking
11	where check_out_date between current_date - 14 and current_date
12	order by check_out_date desc;

Data Output	Messages	Notifications
+ 📄 ▼ 📋 ▼ 🗑️ 🗑️ 📄 ⬇️ 📈 SQL		
QUERY PLAN text 🔒		
1	Sort (cost=1.35..1.35 rows=1 width=114) (actual time=0.215..0.216 rows=2 loops=1)	
2	Sort Key: check_out_date DESC	
3	Sort Method: quicksort Memory: 25kB	
4	-> Seq Scan on booking (cost=0.00..1.34 rows=1 width=114) (actual time=0.181..0.184 rows=2 loop...)	
5	Filter: ((check_out_date <= CURRENT_DATE) AND (check_out_date >= (CURRENT_DATE - 14)))	
6	Rows Removed by Filter: 13	
7	Planning Time: 1.519 ms	
8	Execution Time: 0.239 ms	

Рис. 22: План выполнения запроса до создания простого индекса

Для ускорения выполнения запроса был создан простой индекс по полю occupancy_status таблицы room.

```
create index idx_booking_check_out_date
on hotel.booking (check_out_date);
```

Рис. 23: Создание простого индекса

После создания индекса запрос был повторно проанализирован.

Query

Query History

1

2

3

4

5

6

7

8

9

10

11

explain analyze

select

id,

room_id,

guest_id,

check_in_date,

check_out_date,

status,

total_cost

from hotel.booking

where check_out_date between current_date - 14 and current_date

Data Output

Messages

Notifications

Showing rows: 1 to 8

SQL

QUERY PLAN

text

1

2

3

4

5

6

7

8

Sort (cost=1.35..1.35 rows=1 width=114) (actual time=0.038..0.039 rows=2 loops=1)

Sort Key: check_out_date DESC

Sort Method: quicksort Memory: 25kB

-> Seq Scan on booking (cost=0.00..1.34 rows=1 width=114) (actual time=0.017..0.020 rows=2 loop...

Filter: ((check_out_date <= CURRENT_DATE) AND (check_out_date >= (CURRENT_DATE - 14)))

Rows Removed by Filter: 13

Planning Time: 0.440 ms

Execution Time: 0.055 ms

Рис. 24: План выполнения запроса после создания простого индекса

До создания индекса время построения плана составляло 1.519 ms, а время выполнения запроса — 0.239 ms. После создания индекса время построения плана составило 0.440 ms, а время выполнения уменьшилось до 0.055 ms

Составной индекс для запроса поиска выехавших постояльцев

Для анализа был выбран запрос поиска постояльцев, выехавших за последние две недели. До создания индекса для него был получен план выполнения.

Query	Query History
1	<code>explain analyze</code>
2	<code>select</code>
3	<code> b.id as booking_id,</code>
4	<code> g.last_name,</code>
5	<code> g.first_name,</code>
6	<code> b.room_id,</code>
7	<code> b.check_in_date,</code>
8	<code> b.check_out_date,</code>
9	<code> b.status</code>
10	<code>from hotel.booking b</code>
11	<code>join hotel.guest g on g.id = b.guest_id</code>
12	<code>where b.check_out_date between current_date - 14 and current_date</code>
13	<code> and b.status = 'завершен'</code>
14	<code>order by b.check_out_date desc;</code>

Data Output	Messages	Notifications
Showing rows: 1 to 11		
QUERY PLAN		
text		
1	Sort (cost=9.86..9.86 rows=1 width=378) (actual time=0.032..0.032 rows=1 loops=1)	
2	Sort Key: b.check_out_date DESC	
3	Sort Method: quicksort Memory: 25kB	
4	-> Nested Loop (cost=0.14..9.85 rows=1 width=378) (actual time=0.023..0.025 rows=1 loops=1)	
5	-> Seq Scan on booking b (cost=0.00..1.38 rows=1 width=110) (actual time=0.016..0.018 rows=1 loops=1)	
6	Filter: (((status)::text = 'завершен'::text) AND (check_out_date <= CURRENT_DATE) AND (check_out_date >= (CURRENT_DATE - ...	
7	Rows Removed by Filter: 14	
8	-> Index Scan using pk_guest on guest g (cost=0.14..8.16 rows=1 width=284) (actual time=0.006..0.006 rows=1 loops=1)	
9	Index Cond: (id = b.guest_id)	
10	Planning Time: 0.167 ms	
11	Execution Time: 0.052 ms	

Рис. 25: План выполнения запроса до создания составного индекса

Для данного запроса был создан составной индекс по полям `status` и `check_out_date` таблицы `booking`.

1	<code>create index idx_booking_status_checkout</code>
2	<code>on hotel.booking (status, check_out_date);</code>

Рис. 26: Создание составного индекса

После этого был повторно получен план выполнения запроса.

The screenshot displays the pgAdmin 4 Query Tool interface. At the top, an SQL query is entered in the editor:

```

1  explain analyze
2  select
3      b.id as booking_id,
4      g.last_name,
5      g.first_name,
6      b.room_id,
7      b.check_in_date,
8      b.check_out_date,
9      b.status
10 from hotel.booking b
11 join hotel.guest g on g.id = b.guest_id
12 where b.check_out_date between current_date - 14 and current_date
13       and b.status = 'завершен'
14 order by b.check_out_date desc;

```

Below the editor, the 'Data Output' tab is active, showing the 'QUERY PLAN' for the executed query. The plan details are as follows:

Step	Operation	Cost	Rows	Width	Actual Time	Actual Rows	Actual Loops
1	Sort	9.86..9.86	rows=1	width=378	(actual time=0.026..0.027)	rows=1	loops=1
2	Sort Key: b.check_out_date DESC						
3	Sort Method: quicksort Memory: 25kB						
4	-> Nested Loop	0.14..9.85	rows=1	width=378	(actual time=0.017..0.019)	rows=1	loops=1
5	-> Seq Scan on booking b	0.00..1.38	rows=1	width=110	(actual time=0.010..0.012)	rows=1	loops=1
6	Filter: (((status)::text = 'завершен'::text) AND (check_out_date <= CURRENT_DATE) AND (check_out_date >= (CURRENT_DATE - ...						
7	Rows Removed by Filter: 14						
8	-> Index Scan using pk_guest on guest g	0.14..8.16	rows=1	width=284	(actual time=0.005..0.005)	rows=1	loops=1
9	Index Cond: (id = b.guest_id)						
10	Planning Time: 0.181 ms						
11	Execution Time: 0.046 ms						

Рис. 27: План выполнения запроса после создания составного индекса

До создания индекса время построения плана составляло 0.167 ms, а время выполнения — 0.052 ms. После создания составного индекса время построения плана составило 0.181 ms, а время выполнения — 0.046 ms. PostgreSQL также продолжил использовать последовательное сканирование таблицы `booking`, поскольку объем таблицы недостаточно велик для заметного эффекта от индекса.

После завершения эксперимента созданные индексы были удалены.

3.6 Использование возможностей Query Tool

При выполнении лабораторной работы использовался инструмент Query Tool среды pgAdmin 4. SQL-запросы вводились и запускались в редакторе запросов. Результаты выполнения анализировались на вкладке **Data Output**, где отображался набор строк, возвращаемый запросом. Для просмотра служебной информации о выполнении команд использовалась вкладка **Messages**, а для анализа плана выполнения запросов применялась команда **EXPLAIN**.

ANALYZE. Также Query Tool позволяет экспортировать результаты выборки в формат CSV и просматривать историю выполненных запросов.

4 Выводы

В ходе выполнения лабораторной работы были созданы и выполнены запросы на выборку данных по базе данных PostgreSQL для предметной области «Отель». Были разработаны два представления: для турагентов и для владельца компании. Также были выполнены три запроса на модификацию данных с использованием подзапросов: INSERT, UPDATE и DELETE.

Кроме того, были созданы простой и составной индексы для двух выбранных запросов и проведён анализ плана их выполнения с помощью команды EXPLAIN ANALYZE. Сравнение показало, что при небольшом объеме таблиц PostgreSQL может продолжать использовать последовательное сканирование, однако работа с индексами позволила изучить механизм оптимизации выполнения запросов. Таким образом, цель лабораторной работы была достигнута.